Detecting phishing from trusted accounts

A layered, incident-driven anti-phishing toolkit: SentinelQA, TrustGuard and netguard

Technical write-up. All incident details are anonymized; all test messages, names and ids are fictional. Every number in this document was produced by the code and tests in this repository.

---

1. Summary

Most anti-phishing tooling asks one question: *is this link bad?* In the incident that started this project, the more important question was *is this sender still who they appear to be?* A colleague's chat account was taken over and used to send a short gift-card lure to everyone on its contact list. Because the sender was a known colleague, several people clicked and gave away their credentials on a fake Steam page. The compromise then spread to other accounts that reused the same password. After the hijacked account was removed, the attacker came back under a new account, and nothing asked who they were.

The project answers each stage of that chain with a separate, explainable control:

- **SentinelQA**, a Chrome (Manifest V3) extension, grades the page in front of the user from A to F. It catches look-alike brand domains, fake sign-in pages, rejected certificates and DNS anomalies.
- **TrustGuard**, a chat bot with one platform-neutral engine and thin Discord and Slack adapters, scores messages from *compromised but trusted* accounts. It detects one sender pushing the same lure to many people, turns user reports of private DMs into moderator alerts, and holds returning or look-alike accounts for re-verification.
- **netguard**, a small command-line check, looks for the symptoms of interception on public Wi-Fi: a swapped router, redirected DNS and rejected certificates.
- A **shared rule set** keeps the layers consistent: one file of brand domains, abused domain endings and look-alike character pairs feeds both the extension and the bot.

Applied to the incident's own lure, the extension grades `steamcommunnity.com/gift-card/pay/50` F (70/100), and the bot scores the same text 95/100 and quarantines it. Benign colleague messages score 0, including one that mentions Steam. The approach is heuristic and stated as such: precision and recall on real traffic have not been measured (§11–12).

2. The incident

The incident is reconstructed and anonymized from the author's own observation.

1. **Network.** A colleague used public Wi-Fi while travelling. Days later his Discord account was hijacked. The most likely path is theft of a session token or credentials on the open network; the delay suggests the stolen access was used later rather than immediately.
2. **Spread.** The attacker messaged everyone on the account's friend list. The message was a "Steam gift $50" link styled like `steamcommunity.com/gift-card/pay/50`, with no greeting and no personal detail.
3. **Lure and harvest.** Recipients trusted the familiar name and the money offer. Several clicked and entered their Steam credentials on a look-alike page.
4. **Cascade.** Victims who reused the same email and password elsewhere lost those accounts too.

5. **Re-infiltration.** The company removed the hijacked account from the work server. The attacker joined again with a *new* account and was let in, because there was no re-verification step.
6. **Human factor.** At least one recipient was suspicious and replied asking, in effect, "what is this?". Trust in the sender still won.

Key insight. URL reputation and spam filters mostly judge the *content* or the *link*. Here the sender was a legitimate, long-standing account. Detection therefore has to be behavioural (this account suddenly sends the same money lure to many people) and contextual (the link imitates a brand, and the page asks for a password while claiming to be that brand). It also has to cover the stages before and after the click.

3. Threat model

Stage	Attacker capability	Assumption the defences rely on
Network	Operates or observes an open Wi-Fi network	Cannot obtain a valid certificate for the victim's sites, so interception shows up as certificate errors, DNS anomalies or a changed router
Takeover	Holds a valid session or credentials for a real member	Can send messages indistinguishable at the protocol level from the real user
Lure	Registers cheap or look-alike domains	Domains imitate brand names; many are young
Harvest	Clones a brand's sign-in page	Page title or heading claims the brand; the domain does not belong to it
Persistence	Creates new accounts at will	New accounts are young; names often imitate the removed one

Out of scope: malware on the endpoint, compromise of the chat platform itself, and reading private DMs. Platform APIs do not allow the last one (§8.5), and the project does not try to work around that.

4. Architecture

Each component answers one or more attack stages:

Attack stage	What happened	Control	Component
Network	Session or credentials stolen on open Wi-Fi	Certificate-error capture; DNS-over-HTTPS cross-check; router hardware-address change; DNS answers redirected to private addresses	Extension, netguard
Spread	Hijacked account mass-messages a lure	Message scoring; same-message burst detection; multi-reporter escalation; edit re-scoring	TrustGuard
Lure link	Look-alike Steam gift-card URL	Brand look-alike and brand-in-domain detection; masked-link detection; domain age; optional Safe Browsing	Extension, TrustGuard
Credential harvest	Fake Steam sign-in page	Password form whose title or heading claims a brand the domain does not own	Extension
Re-infiltration	Attacker rejoins with a new account	Holds for removed ids, look-alike names, very new accounts, and a post-compromise lockdown; state survives restarts	TrustGuard

The layers follow one design rule: **the logic is pure, and the platform code is thin.**

- In the extension, all scoring is in `lib/core.js`, which makes no `chrome.*` calls. The popup, content script and service worker load the same file, and Node unit tests require it.

- In the bot, `trustguard/core` has no third-party dependencies. Each adapter only converts a platform event to a neutral `Message` or `JoinEvent`, calls the engine and acts on the verdict.
- In netguard, parsing and decisions (`checks.py`) are separate from the probes that run commands or open connections (`probes.py`).

The rule pays off three ways. Each part can be tested without a browser, a chat workspace or a real network. Every verdict can list exactly which signals produced it. And adding a platform means writing an adapter, not a new detector.

5. SentinelQA browser extension

5.1 Data flow

```
service worker (background.js)            content script (content.js)
  response headers  ─┐                      password forms, mixed content,
  certificate errors ├─► per-tab record ◄─  brand claim on a login page
  redirect hops      │   (session storage)
  connected IP      ─┘           │
                                  ▼
              popup: URL checks + record + optional online lookups
                     ─► score ─► grade A–F with itemised reasons
```

Records are keyed by tab id, so one page never sees another page's results, and they are deleted when the tab closes. Updates for a tab are queued so that concurrent listeners cannot overwrite each other. All findings are rendered with `textContent`. The original version inserted the raw URL with `innerHTML`, a self-inflicted injection risk in a security tool, and that was the first thing fixed.

5.2 Checks and weights

Level	Check	Points
URL	No HTTPS (local development hosts exempt)	25
URL	Raw IP address as host	30
URL	`@` in the URL (the real host follows it)	20
URL	Punycode (`xn--`) host, possible homoglyphs	25
URL	Abused domain ending (shared rule set)	15
URL	Digits in the domain	5
URL	Pressure words (login, verify, suspended…)	10 each, max 20
URL	URL longer than 75 characters	10
URL	Look-alike of an official brand domain	55
URL	Brand name inside an unrelated domain	35
URL	Brand + reward lure together	15
Page	Each page finding (password over HTTP, mixed content, fake login)	35
Transport	Certificate/TLS error recorded for the tab	70 (always F)
Transport	Each missing security header	8
Transport	Redirect chain of ≥3 hops or ≥3 domains	10

Level	Check	Points
Online	Domain registered < 30 days / < 180 days ago	30 / 10
Online	Server IP not among DNS-over-HTTPS answers	10
Online	Google Safe Browsing match	100

Grades: A < 10, B < 30, C < 50, D < 70, F ≥ 70. The score is capped at 100.

5.3 Brand impersonation

The original URL checks would have scored the incident link low: it uses HTTPS, an ordinary `.com` ending, no IP address and no `@`. The missing piece was brand awareness. Two rules, run against a list of commonly impersonated brands and their official registrable domains:

- **Look-alike.** First undo common character swaps (`rn→m`, `vv→w`, `0→o`, `1→l`, `3→e`, `5→s` …). Then compare the domain's main label with each official label by edit distance. The tolerance scales with length: 0 for labels under 6 characters, 1 for 6–7, 2 for 8 or more. Without that scaling, ordinary words like `stream` would match `steam`, and a unit test guards that case.
- **Contains.** The brand name appears as a token of a domain the brand does not own (`steam-giftcard.xyz`).

On the page, the content script compares the page's *own* identity (title, `og:site_name`, first heading) with the domain whenever there is a password field. It deliberately ignores "Sign in with Google" buttons and partner logos elsewhere on the page, which would otherwise cause false alarms on legitimate sites.

5.4 Transport and online checks

The service worker records `ERR_CERT_*` and `ERR_SSL_*` failures for main-frame requests. It keeps them on the tab's report even if the user clicks through the warning, because that is when the information matters most. It also records redirect hops and the IP the browser actually connected to.

Online lookups run only when the user opens the popup, and only for the site being scanned, so the extension never streams browsing history to a third party:

- **RDAP** (via rdap.org) gives the registration date. Young domains score higher.
- **Cloudflare DNS-over-HTTPS** gives the addresses a trusted resolver returns. If the connected IP is not among them, the score rises by 10. This is a weak signal because CDNs and geo-DNS legitimately differ.
- **Google Safe Browsing** is used only with the organisation's own API key.

Results are cached locally, failures are skipped rather than fatal, and all lookups can be switched off in Settings.

6. Shared rule set

Brand domains, abused domain endings, link shorteners, multi-part suffixes (`co.uk`) and look-alike character pairs live in one file, `shared/rules.json`. `npm run build:rules` generates the extension's copy (`lib/rules.js`) and the bot's (`shared_rules.json` inside the Python package), and a test fails when either copy is stale.

This came out of an actual defect. When the bot was written, its rule table was copied from the extension and then extended. By the time of the audit the bot recognised Slack impersonation and the `.gift` ending and the extension did not. The same link could get a different verdict depending on where it was opened. Word lists that are genuinely layer-specific stay local: URL pressure words in the extension, chat greetings and urgency phrases in the bot.

7. netguard: the network stage

The extension sees HTTP(S) only; it has no packet-level visibility and does not claim any. netguard covers the Wi-Fi stage from the client side with three passive checks:

Check	Method	Severity
Router changed	Trust-on-first-use: the router's hardware address, read from the OS ARP cache, is recorded per network on first visit. A different one later on the same network name and gateway IP is how an evil-twin hotspot or ARP spoofing looks to the client	critical
DNS redirected	A probe host resolves to a private address locally but to public addresses over DNS-over-HTTPS	critical
DNS disagreement	Local and DNS-over-HTTPS answers share no /16 block	warning
Certificate rejected	A TLS connection to a major site (default: Slack, Microsoft and Google sign-in) fails verification	critical

Exit codes (0 ok, 1 warning, 2 critical) let it run from a login script. It sends no ARP packets, does no scanning and captures nothing. Active techniques were kept out of scope deliberately: they need elevated privileges, behave differently on every OS, and add little over these symptoms.

The first live run on a normal office network returned a false warning. The local resolver and Cloudflare gave different addresses for `login.microsoftonline.com`, from the same /16 block, because the service is load-balanced. Comparing answers per /16 block removed the warning without weakening the "private address for a public site" check, which is the one that matters.

8. TrustGuard bot

8.1 Engine

The engine takes a normalized `Message` and returns a `Verdict`: allow, warn (score ≥ 35) or quarantine (≥ 60), with itemised reasons. Both thresholds can be changed through environment variables, so a pilot can start in warn-only mode.

Signal	Points
External link (domain not on the organisation's trusted list)	20
Money / gift / reward lure, or a currency amount	25
Link domain is a look-alike of a brand / uses a brand name	40 / 30
Masked link: visible text shows one domain, the link opens another	30
Short (≤ 25 words), greeting-less message with a link	10
Urgency wording; link shortener; abused domain ending; @everyone / @here	10 each

Signal	Points
Sender account under 7 days old (with a link)	10
Sender with history shares a domain never used before	10
Same message to ≥ 3 destinations within 10 minutes	35

The "no greeting" signal comes straight from the incident: messages between colleagues usually open with a greeting, and the lure did not. The greeting list includes the Bangla and Vietnamese forms seen in the team.

8.2 Spread detection

A burst is one sender sending the *same* message to several destinations in a short window. "Same" is decided by a fingerprint that ignores case, whitespace and numbers, so "$50" and "$100" variants still match. This is the bot-side equivalent of the incident's mass DM.

8.3 Reports: the private-DM gap

No Discord or Slack bot can read private DMs between two users, and the incident spread through exactly those. The bot therefore treats reports as the DM-side signal. A recipient forwards (Discord) or uses *Share message* (Slack) to send the suspicious DM to the bot. The bot replies with a verdict and recovery steps. When two *independent* reporters name the same sender within 30 minutes, moderators get a "possible compromised account" alert. Repeat reports from one person do not count twice.

8.4 Re-verification

A join is held for an admin when any of these hold: the user id was removed or marked compromised before; the name closely resembles a removed account (after Unicode normalization and the same look-alike swaps used for domains); the account is under 7 days old; or the community is in the 72-hour lockdown that follows marking an account compromised. An admin releases the hold only after confirming identity out-of-band, by call or in person.

This is the control that was missing in the incident. Because it must not be lost on restart, removals, approvals and the lockdown are persisted to SQLite. The store holds ids, names and timestamps only, never message text, and a test inspects the database to confirm that.

8.5 Adapters and platform reality

Platform	Status	What the bot can see and do
Discord	Implemented	Server channels (Message Content intent) and DMs to the bot. Quarantine deletes the message, times the sender out for one hour and alerts moderators. A hold role restricts held joins. Bans are recorded automatically
Slack	Implemented; the office deployment	Channels it is invited to and DMs to the bot. Bot tokens cannot delete other users' messages or deactivate accounts, so quarantine means a threaded warning plus a moderator alert for an admin to act on. Deactivations are recorded automatically. Joins to protected channels (including Slack Connect guests) are checked; the bot can optionally remove held people. Installs from a manifest with minimal scopes over Socket Mode (no inbound endpoint)
Teams	Future work	Bot Framework

Platform	Status	What the bot can see and do
Telegram, WhatsApp	Out of scope	Bots see only messages sent to them / closed API

Both adapters re-score edited messages, so a harmless message cannot be turned into a lure after people have read it. Links quoted in alerts are defanged (`hxxps://evil[.]example`). Slack auto-links bare domains, so on Slack the reason text is defanged too. That was found by a test, not by assumption.

The browser extension complements the bots: it works wherever the link is opened in a browser, whatever chat app delivered it. In short, the extension protects everywhere, and the bots protect where they are invited.

9. Privacy

The tools exist to protect colleagues, so they must not expose them.

Component	Reads	Stores	Sends out
Extension	URL, headers and page structure of the active tab	Per-tab results in session storage (cleared on tab close); cached lookup results	Only on popup open, for that site: hostname to rdap.org and Cloudflare; full URL to Safe Browsing if a key is set. Can be switched off
TrustGuard	Messages in channels it was added to; DMs sent to it	Removed and compromised ids, names, approvals, lockdown end (SQLite). Burst and report counters in memory only	Alerts inside the chat workspace; quarantine alerts quote up to 500 characters with links defanged
netguard	OS ARP, route and Wi-Fi tables	Hashed network id, router hardware address and date, on the user's machine	TLS connections and DoH lookups for three fixed probe hosts (DoH optional)

None of the components logs conversations, records passwords or form contents, or sends browsing history anywhere. Test fixtures are fictional. Before the office rollout, the guide requires a security review, notice to colleagues (with counsel confirming the lawful basis), and approval of the third-party lookup services (`docs/rollout-guide.md`).

10. Worked examples

These outputs come from running the current code.

The incident link in the extension. `https://steamcommunnity.com/gift-card/pay/50` scores 70, grade **F**: look-alike of `steamcommunity.com` (55) plus brand-and-reward-lure combination (15). The URL-only version of the extension would have graded this link A, since it is HTTPS on a `.com` domain. A variant, `https://steam-giftcard.xyz/claim`, scores 65 (D): abused ending (15), brand in an unrelated domain (35) and lure (15). The real `https://store.steampowered.com/` scores 0 (A).

The fake sign-in page. The bundled test page served over HTTP from a non-local name reaches 100 (F). It combines no HTTPS, the brand name in the domain, the reward lure, the password-over-HTTP page finding, the fake-login page finding and missing headers.

The incident message in the bot. The bare text `steamcommunnity.com/gift-card/pay/50` scores **95** and is quarantined: external link (20), money lure (25), no greeting (10), brand look-alike (40). The same lure from one sender to three DMs within three minutes reaches the 100 cap with the burst signal (35). Two benign messages score 0: "hey, standup moved to 10:30 today" and "Hello! Steam summer sale starts tomorrow, anyone want to split a bundle?" The second mentions the brand but has no link and a greeting.

Re-infiltration. After `!tg compromised` on a fictional account `rahim.dev`, a new account named `rahim_dev` joining five days later is held because its name is a look-alike, even though the 72-hour lockdown has ended. Within the lockdown, every new join is held. Both holds still apply after the bot restarts.

11. Evaluation

Suite	Tests	What is covered
Extension (Node)	25	URL heuristics (clean site, HTTP, raw IP, `@`, punycode, abused ending, keyword cap, long URL), the incident link → F, false-positive guards (`stream.com`, official subdomains), suffix handling, record merging (headers, page findings, certificate errors, redirects), online signals, rule-copy parity
TrustGuard (Python)	43	Six fictional lure messages flagged and six benign ones passed; incident link quarantined; burst and window expiry; reports and escalation; masked links; re-verification incl. look-alike rejoin and lockdown; persistence across restart with no message text stored; Slack adapter (reports, commands, edits, channel joins) and Discord adapter (quarantine, edits, forwarded messages) against fake clients
netguard (Python)	10	Parsers on captured Windows, macOS and Linux output; router change, DNS and certificate decisions; an end-to-end first-visit-then-swapped-router run

CI runs all suites on every push, netguard on all three operating systems. Beyond unit tests:

- The Slack adapter was driven through the real Bolt 1.30 dispatcher.
- The Discord adapter was exercised with discord.py 2.7.
- netguard was run live: a clean network reported OK, and a test site with an expired certificate was flagged as critical.

What this evaluation is not. It is functional testing on fictional data modelled on one incident. There is no labelled corpus of real messages or URLs, so precision, recall and false-positive rates are unknown. The office pilot (§13) is designed to produce that data.

12. Limitations and future work

Area	Current limitation	Future direction
Risk scoring	Fixed weights; no evasion handling, class-imbalance treatment or measured false-positive rate	ML-based scoring with evasion testing on labelled data
Bot anomaly detection	Public keyword and rate rules can be written around (add a greeting, avoid lure words)	Per-sender statistical baselines; formal threat model
Token theft on Wi-Fi	Symptoms detected (certificate errors, DNS redirection, router change), not the theft	Protocol-level analysis of session handling
Fake login detection	Brand list + form heuristics	Auth-flow analysis (OAuth/SAML failure modes)
Re-verification	Admin approval after an out-of-band check; pragmatic, not formally derived	Formal access-control / zero-trust identity assurance

Area	Current limitation	Future direction
User behaviour	People click despite doubt	Warning UX design and evaluation
Base rates	With few true phishing messages among many benign ones, even a low false-positive rate means most alerts are false	Threshold selection grounded in measured rates

13. Deployment

The office rollout (`docs/rollout-guide.md`) runs in this order:

1. **Approvals:** security review and change record, notice to colleagues, and approval of the third-party lookup services.
2. **Extension:** pilot via developer mode, then force-installed through Chrome Enterprise policy.
3. **Bot:** the Slack bot installed from the manifest, with a private moderators channel and the persistent state file, starting in warn-only mode by raising the quarantine threshold.
4. **netguard:** offered to colleagues who travel.

During the pilot, moderator feedback on each alert, whether it was a true or false positive, is the first real measurement of the heuristics.

Appendix: reproducing the results

```
npm test
```

```
cd trustguard && python -m unittest discover -s tests -v
```

```
cd netguard && python -m unittest discover -s tests -v
```